
Partner Integration Guide

ResidenceVertical — Property Report API

Referral links & server-minted checkout links

Report API, webhooks, commissions and weekly settlement

Version 1.4 · August 2026

ResidenceVertical Partner API — integration guide (v1)

This guide is for developers and product owners at a partner company (real-estate portal, agency, broker network) who want to put ResidenceVertical premium property reports in front of their own users. There are **two ways to do that**, and both can run at the same time on one partner account:

- **Referral mode (recommended)** — you send your users to **our** checkout, where they buy the report at the normal 50 lei price and we handle payment, invoicing and delivery. You earn a commission for every report that generates successfully, **paid out to you weekly**. Two tiers: server-minted **checkout links** (recommended — one API call, exact attribution, per-lead tracking; §2.1) or a **zero-code link** you simply place on your site (§2.2). Start at §2.
- **API mode** — for partners **embedding reports in their own product**: your backend calls our API server-to-server with an API key, you charge your own user at your own price, and we invoice you weekly for the generated reports — with your commission as a visible negative line on the invoice. Start at §3.

If you only need the machine-readable contract, the OpenAPI document is served at `GET /api/partner/v1/openapi.yaml` on every environment (no authentication).

1. Overview — the two integration modes

	Referral mode (recommended)	API mode
What you build	A checkout link minted with one server call (recommended, §2.1) — or a zero-code link on your site (§2.2)	A server-to-server backend integration
Who your user pays	ResidenceVertical — the normal 50 lei checkout	You , at your own price; we never charge your user
Invoice to the end buyer	Ours (the standard buyer invoice)	Yours
Report delivery	Our standard flow: delivery email + the report page	You hand over the report (<code>viewUrl</code> and/or the PDF)
What you earn	<code>commissionPct</code> × 50 lei per generated report	<code>commissionPct</code> of the list price, given as a visible negative line on our weekly invoice to you
Money direction	We pay you — a weekly SEPA payout to your IBAN (§2.4)	You pay us — a weekly invoice, net of your commission (§16)
Integration effort	One server call per lead (checkout links) — or ~zero with the static link	API key handling, create/poll or webhooks, <code>viewUrl</code> /PDF handling

Ground rules shared by both modes:

- **Only successfully generated reports count** (`status: generated`). A failed report earns no commission in referral mode and is never billed in API mode.
- **One account, one commission percentage.** `commissionPct` (default 15%, configured by ResidenceVertical per your agreement, visible on `GET /me`) applies to both modes.
- **Both modes can run simultaneously** on the same account (§2.7). Each report belongs to exactly one mode: requests your backend creates through the API are API mode; purchases your users complete on our checkout after following your link are referral mode. The two money flows stay separate — they are not netted against each other.
- Days, weeks and months are counted in the `Europe/Bucharest` time zone.

Every referral partner receives an **API key** at onboarding. It authenticates the checkout-link mint (§2.1) and the tracking endpoints `GET /me` and `GET /referrals` (§2.5, §2.6); the buyer's purchase itself never involves it. Even a zero-code, link-only partner keeps the key for tracking. Key-handling rules are in §5.

2. Referral mode — send your users to our checkout (recommended)

Referral mode is the flagship integration: your site hands the user over to the ResidenceVertical checkout, the user completes the purchase like any direct buyer, and you earn your commission for every report that generates successfully. You never touch a payment, an invoice or a PDF. There are **two tiers**, and both feed the same referral ledger, the same commission and the same weekly payout:

- **Checkout links — the recommended tier (§2.1).** Your backend mints a link for a specific property with **one API call**; the buyer lands on our checkout with the address already filled in. Attribution is exact and server-side, and your own lead id (`externalReference`) comes back per conversion on `GET /referrals`.
- **The zero-code link (§2.2).** The static `/p/<your-slug>` URL you simply place on your site. No backend at all; attribution rides on the visitor's browser.

The flow, end to end (either tier):

1. Your site sends the user to ResidenceVertical — through a checkout link you minted (§2.1), or through your static referral URL (§2.2).
2. Our page stores your attribution in the user's browser (valid **30 days**) and takes them into the normal report flow — under a discreet co-branding chip, "*Comandă prin partener: <your account name>*", on the order form and the payment summary, so the buyer sees the hand-off came from you without being interrupted by it.
3. The user pays the full **50 lei** on our checkout (card, via Stripe). We issue our standard buyer invoice and deliver the report exactly as for a direct customer — delivery email plus the interactive report page.
4. When the report reaches `generated`, your commission is **earned**: `round(5000 × commissionPct / 100)` bani per report — 750 bani = 7,50 lei at the default 15%.
5. Every Monday we close the previous week and **pay the week's earned commissions to your IBAN** in one SEPA transfer (§2.4). You invoice us for the commission.

2.1 Checkout links — the recommended tier

One server call mints a **checkout link** for one property; the returned `url` goes behind the button on your site ("Raport ResidenceVertical" next to the listing, or in the lead email). When the buyer opens it, our checkout starts with the address — and, if you sent one, the buyer's email — already filled in, under your co-branding chip. This is the tier to build if you have any backend at all: attribution does not depend on what the visitor's browser happens to have stored, and every conversion carries your own lead id back to you.

Mint a link (server-side, with your API key — the same key rules as §5):

```
curl -sS -X POST "https://gamma.residencevertical.ro/api/partner/v1/checkout-links" \
-H "Authorization: Bearer $RV_PARTNER_KEY" \
-H "Content-Type: application/json" \
-d '{
  "address": { "street": "Strada Turda", "streetNumber": "94", "city": "București", "postal
  "propertyType": "apartment",
  "externalReference": "lead-84213",
  "customer": { "email": "buyer@example.com" },
  "expiresInHours": 48
}'
```

Response **201 Created** :

```
{
  "checkoutLinkId": "pcl_Ab3dE9f1Gh2jK4mN6pQ8rS0tU1vW3xY5",
  "url": "https://gamma.residencevertical.ro/c/pcl_Ab3dE9f1Gh2jK4mN6pQ8rS0tU1vW3xY5",
  "expiresAt": "2026-08-20T10:00:00Z",
  "externalReference": "lead-84213"
}
```

`url` is the deliverable — hand it to the browser as-is (it carries no key; the opaque token is the whole link).
`checkoutLinkId` is the same token, for your own records.

Field	Required	Rules
<code>address.street</code> / <code>address.streetNumber</code> / <code>address.city</code>	yes	The same address rules as <code>POST /reports</code> (§7.1) — name-only street, building number, locality.
<code>address.county</code> / <code>address.postalCode</code>	no	Same as §7.1.
<code>propertyType</code>	yes	<code>apartment</code> or <code>house</code> .
<code>externalReference</code>	no — but send it	Your lead/order id, ≤ 128 chars. Echoed on the link and, once the buyer converts, on that referral's row in <code>GET /referrals</code> (§2.6) — this is your per-lead conversion tracking .
<code>customer.email</code>	no	RFC-shaped. Pre-fills the buyer's email on our checkout; nothing else happens with it (minting a link sends no email).
<code>expiresInHours</code>	no	Integer, 1..720 (up to 30 days); default 48 .

No `coordinates` field — the buy flow geocodes the address exactly as it does for a direct buyer.

How a checkout link behaves:

- **Reusable until expiry, by design.** A link is never consumed by being opened: reloads, the back button or a second look days later must never brick the buyer. Per-lead uniqueness comes from your `externalReference`, not from a one-shot token — mint one link per lead and reuse it in that lead's context.
- **Expiry is yours to choose** (1 hour to 30 days, default 48 h). An expired link shows the buyer a calm message — *"Linkul de comandă nu mai este valid. Cere partenerului un link nou."* — with a button that continues to the plain order form, so it is never a dead end: the sale can still happen, only the pre-fill and the exact attribution are lost. Mint a fresh link when a lead re-engages after expiry.
- **Attribution is exact.** The link was minted on your account, so a purchase completed through it is attributed to you server-side, and the referral row copies the link's `externalReference` and `checkoutLinkId` (§2.6). The 30-day browser attribution window (§2.2) applies on top: a buyer who opens your link, leaves, and buys later within the window is still yours.
- Everything downstream is the standard referral economics: 50 lei paid to us, commission earned on `generated` only (§2.3), weekly payout (§2.4).

!!! note " `GET /checkout-links/{token}/resolve` — the page calls this itself" Opening a checkout link, our page resolves the token through a **public** endpoint — no API key involved; the token is the whole credential:

```

` ``
GET /api/partner/v1/checkout-links/{token}/resolve
` ``

`200` returns what the page pre-fills: `{ partnerSlug, partnerName, address{street,
streetNumber, city, county, postalCode}, propertyType, customerEmail, expiresAt }`. An
expired link answers `410 checkout_link_expired`; a token that never existed,
`404 not_found`. You never need to call it — it is documented because it is part of
the contract (and the local mock implements it, §6, so you can watch the whole loop
run before you have a key).
```

2.2 The zero-code link (`/p/<your-slug>`)

No backend? The static referral URL still works, and it is genuinely zero-code. Read it from `GET /me` (§2.5) — the `referral.referralUrl` field, e.g. `https://residencevertical.ro/p/agentia-exemplu`. Copy it as-is; do not construct variations of it yourself (the slug is assigned by us, and a mistyped slug silently earns you nothing).

How attribution works:

- Opening the link stores your partner slug in the visitor's **browser** and immediately continues into the normal report flow on our site. The attribution lives **30 days**; a newer partner link overwrites an older one (**last click wins**).
- The attribution is stamped on the payment **at purchase time**. If your user buys within the 30-day window — same visit or three weeks later — the report is yours.
- **The link can never break a sale.** An unknown or suspended slug, an expired attribution or a browser that cleared its storage simply loses the attribution — the user's checkout works regardless.

- Attribution requires the purchase to happen in the same browser the link was opened in; a user who switches devices between click and purchase is not attributed. Place the link where the buying intent is (next to the property, in the lead email), not only on a landing page.

Your buyer sees the same discreet co-branding chip as on the checkout-link tier — *"Comandă prin partener: <your account name>"* — on the order form and the payment summary, so the hand-off is visible without interrupting the purchase.

Pre-filling the property address

You can append query parameters to the referral link and our landing pre-fills the report form with them — one less step for your user, and fewer typos in the address:

Parameter	Meaning
<code>street</code>	Street name, without the number (e.g. <code>Strada Turda</code>).
<code>number</code> (or <code>streetNumber</code>)	House/building number (e.g. <code>94</code> , <code>12B</code>).
<code>city</code>	Locality (e.g. <code>București</code>).
<code>type</code> (or <code>propertyType</code>)	<code>apartment</code> or <code>house</code> .
<code>postalCode</code>	Postal code (e.g. <code>011322</code>).

```
https://residencevertical.ro/p/agentia-exemplu?street=Strada%20Turda&number=94&city=Bucure%C8
```

URL-encode the values (Romanian diacritics included). Everything is optional and the user can still edit it all — the parameters only pre-fill the form. Unknown parameters are ignored, and so is a `type` outside `apartment` / `house` (the user simply picks it).

Send `type` whenever you know it. Our form asks for the property type first and only reveals the address fields once it is chosen, so a link without `type` shows your user the type picker rather than the address you pre-filled. `street` and `city` are the minimum for any pre-fill to happen at all.

The parameters are a best-effort convenience; a **checkout link** (§2.1) pre-fills the same form server-side, attributes exactly, and adds per-lead tracking — prefer it whenever you have a backend to mint from.

2.3 What your user experiences — and what you earn

Your user is **our customer** for the transaction: our page, our 50 lei price, our card checkout, our invoice to them, our delivery email, our report page, our support and our refund handling. You are outside the payment and delivery path entirely, and **no buyer data flows back to you** — the referral ledger you can read (§2.6) deliberately carries no buyer email or any other personal data.

Commission is earned per report that reaches `generated`, at the `commissionPct` snapshot when the purchase was attributed (a later percentage change never rewrites history). A referral that never turns into a generated report earns nothing: the report failed terminally (the buyer is refunded by our standard flow), the payment was refunded, or no report materialised for the payment. You see each referral's state in `GET /referrals` (§2.6): `pending` → `earned`, or `void` for the cases that will never earn.

2.4 The weekly payout — we pay you

- A payout week runs **Monday 00:00 → Sunday 24:00, Europe/Bucharest** — the same weekly cycle as API-mode settlements (§16).
- **Every Monday** we close the previous week: all your commissions earned up to that Sunday that have not been paid yet are bundled into one payout. A week with nothing earned produces no payout row — nothing is lost, the next close picks up whatever is waiting.
- We pay the payout total to **your IBAN** by SEPA credit transfer. The remittance line names the period and your slug, so your bank statement is self-explanatory.
- The payout is **gross and B2B**: we withhold nothing. You issue **your commission invoice to ResidenceVertical** for the paid amount — the weekly statement email tells you the exact figure and period to invoice, so your accounting has a document trail on both sides.
- **No IBAN on file, no payout.** Earned commissions wait (they do not expire) until an IBAN and account-holder name are configured on your account — give them to us at onboarding, or whenever they change.
- **Refunds.** A buyer refund before the week closes voids that referral (it is simply not paid). A refund that lands after a payout was already paid is settled manually — we contact you and typically deduct it from the next payout.

Your running totals are always visible on `GET /me` (§2.5): pending, earned-but-unpaid, and everything paid out to date.

2.5 The `referral` block on `GET /me`

`GET /me` (§7.7) carries a `referral` object for every account:

```
"referral": {
  "referralUrl": "https://gamma.residencevertical.ro/p/agentia-exemplu",
  "commissionPct": 15,
  "pendingCents": 750,
  "earnedUnpaidCents": 1500,
  "paidCentsAllTime": 2250
}
```

Field	Meaning
<code>referralUrl</code>	The zero-code link to put on your site (§2.2), on this environment's host. Checkout links (§2.1) are minted per lead instead of read from here.
<code>commissionPct</code>	Your percentage — the same value as the top-level field.
<code>pendingCents</code>	Commission on attributed purchases whose report is still generating (bani). Not earned yet.
<code>earnedUnpaidCents</code>	Earned commission waiting for its weekly payout (or sitting in a payout not yet paid).
<code>paidCentsAllTime</code>	Total commission already paid out to you, ever.

2.6 `GET /api/partner/v1/referrals` — your referral ledger

Newest first. `limit` ≤ 100 (default 20), `offset` default 0 — the same pagination and envelope as `GET /reports` (§7.6). Requires your API key (§5).

```
{
  "items": [
    {
      "referralId": "5b8e2f1a-7c3d-4e9b-a1f6-0c4d8e2b6a90",
      "createdAt": "2026-08-18T09:41:00Z",
      "status": "earned",
      "priceCents": 5000,
      "commissionCents": 750,
      "earnedAt": "2026-08-18T09:45:12Z",
      "paidAt": null,
      "externalReference": "lead-84213",
      "checkoutLinkId": "pcL_Ab3dE9f1Gh2jK4mN6pQ8rS0tU1vW3xY5"
    }
  ],
  "count": 1,
  "limit": 20,
  "offset": 0
}
```

Field	Meaning
referralId	The referral's id — quote it in support conversations.
createdAt	When the attributed purchase was confirmed.
status	pending (paid, report generating) → earned (report generated — commission owed to you), or void (never earns: the report failed, the payment was refunded, or no report materialised).
priceCents / commissionCents	The report's list price (5000 = 50 lei) and your commission on it, snapshotted at attribution time.
earnedAt	When the report reached generated ; null for pending / void .
paidAt	When the weekly payout containing this referral was paid; null until then. status stays earned — paidAt is the paid marker.
externalReference	Your own lead id, copied from the checkout link (§2.1) the buyer converted through — the per-lead conversion tracking : match it against the ids you minted with. null when the buyer came through the zero-code link (§2.2).
checkoutLinkId	The checkout link (pcl_...) that attributed this purchase; null for zero-code attributions.

Deliberately absent: any buyer data. The buyer is our customer; you get the money trail, not the person.

2.7 Mixing referral mode with API mode

One account can do both at once, at the same commissionPct . Each report belongs to exactly one mode — API-created requests settle on our weekly invoice **to you** (§16); referred checkout purchases pay out on our weekly SEPA transfer **from us** (§2.4). The two flows are independent and are **not netted** against each other: in a busy week you may both receive a payout and owe an invoice. The weekly statement email covers both — its referral section appears whenever you had referral activity that week.

!!! note "Testing referral mode" On the test environment the checkout pages (/c/<token> and the /p landing included) sit behind our team access wall (unlike /api/partner , which is open — §14 has the details), so an end-to-end referral test on gamma.residencevertical.ro is something we run **together** at onboarding. Everything under /api/partner works with your test key at any time: minting checkout links, the public resolve, and the tracking endpoints (GET /me , GET /referrals). The local mock (§6) covers the whole loop — mint, resolve, a stand-in /c/<token> landing, and canned referral data — so you can build your integration and reconciliation first.

3. API mode overview — embed reports in your own product

API mode is for partners who embed the report **inside their own product**: your backend calls our API with an API key, we generate the report, and you hand your user the interactive report **web page** — with the PDF available on it as an optional export. You charge your own user at your own price; weekly, we invoice you for the generated reports with your commission as a visible negative line (§16). Everything from here through §16 is about this mode — though §4 (environments) and §5 (key handling) apply equally to a referral partner's tracking calls.

A typical "Generate report" button works like this:

```
sequenceDiagram
    autonumber
    participant U as End user (browser)
    participant P as Partner backend
    participant RV as ResidenceVertical Partner API
    participant W as Partner webhook endpoint

    U->>P: Click "Raport ResidenceVertical" for a property
    P->>RV: POST /api/partner/v1/reports (Bearer key, Idempotency-Key)
    RV-->>P: 202 Accepted { reportRequestId, status: "processing" }
    P-->>U: "Report is being generated (~4 min)"
    alt Polling
        loop every 5-10 s, up to 6 min
            P->>RV: GET /api/partner/v1/reports/{reportRequestId}
            RV-->>P: { status: "processing" | "generated" | "failed" }
        end
    else Webhook
        RV->>W: POST report.generated / report.failed (signed)
        W-->>RV: 2xx
    end
    Note over P,RV: The deliverable is viewUrl or the report web page.<br/>downloadUrl (PDF) :
    P-->>U: "Vezi raportul" or a link to viewUrl
    U->>RV: GET /raport/{reportRequestId}?t=... (browser, no API key)
    RV-->>U: The interactive report web page (offers the PDF too)
    opt Optionally, download the PDF yourself
        P->>RV: GET /api/partner/v1/reports/{reportRequestId}/pdf (Bearer key)
        RV-->>P: application/pdf (bytes)
        P-->>U: Serve / email the PDF
    end
end
```

Key properties of the flow:

- Report generation is **asynchronous**. `POST /reports` returns immediately with a `reportRequestId`; the report is usually ready within about 4 minutes.
- Every **successfully generated** report is billable to your account at the list price (50 lei); we settle **weekly**, with your commission given as a visible negative line on our invoice (see §16). Failed requests are never billed. What you charge your own user — at your own price — is entirely your business.

- **The deliverable is the report web page.** A generated report carries `viewUrl` — a signed, time-limited link to the same interactive report ResidenceVertical's own buyers read. You hand it straight to your user; their browser opens it and no API key is involved (§14). The page offers the PDF download itself, so this one link gives your user everything.
- `downloadUrl` is the optional **PDF export** of the same report — fetched server-to-server with your key and served to your user by you (§13), for flows that need the file itself (archiving, attaching to your own email). Many integrations never call it at all.

The recommended integration is therefore a **"Vezi raportul" button on** `viewUrl`; the PDF endpoint is optional.

4. Environments

Purpose	Base URL	Notes
Integration / testing	<code>https://gamma.residencevertical.ro</code>	Test environment. Reports are real (generated by the full pipeline), but data may be reset without notice and the environment may be unavailable during maintenance windows. Use it for building and testing your integration.
Production	<code>https://residencevertical.ro</code>	Enabled for your account after go-live sign-off with ResidenceVertical.

API keys are **environment-specific**: a key issued for the test environment starts with `rvp_test_` and a production key starts with `rvp_live_`. A test key never works in production and vice versa. Partner accounts, usage counters and webhook settings are also per environment.

All examples below use the test base URL. Every path is prefixed with `/api/partner/v1`.

5. Authentication and key handling

Every endpoint except `GET /openapi.yaml` requires:

```
Authorization: Bearer rvp_test_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Rules:

- **Server-side only.** The key must live in your backend's secret store. Never embed it in browser JavaScript, mobile apps, HTML, public repositories or client-side config — anyone holding the key can generate billable reports on your account.
- **HTTPS only.** Both environments are HTTPS; plain HTTP is not served.

- **Rotation.** ResidenceVertical can issue several keys per account (for example one per server or per staging/production deployment). Ask for a new key, switch your servers over, then ask us to revoke the old one — there is no downtime.
- **Revocation.** A revoked or expired key returns `401 unauthorized` immediately. If you suspect a leak, contact support at once so we can revoke it.
- The full plaintext key is shown to ResidenceVertical staff **once** when it is created and handed to you through a secure channel; we only store a hash and the first 16 characters (`rvp_test_AbCd123`) for display. Keep your own copy safe.

Every response, success or error, carries an `X-RV-Request-Id` header. Log it next to your own request id — support will ask for it.

6. Quick start (curl)

Create a report request:

```
curl -sS -X POST "https://gamma.residencevertical.ro/api/partner/v1/reports" \
-H "Authorization: Bearer $RV_PARTNER_KEY" \
-H "Content-Type: application/json" \
-H "Idempotency-Key: lead-84213" \
-d '{
  "address": { "street": "Strada Turda", "streetNumber": "94", "city": "București", "postal": "110100" },
  "propertyType": "apartment",
  "customer": { "email": "buyer@example.com", "name": "Ion Popescu" },
  "externalReference": "lead-84213"
}'
```

Poll the status:

```
curl -sS "https://gamma.residencevertical.ro/api/partner/v1/reports/$REPORT_REQUEST_ID" \
-H "Authorization: Bearer $RV_PARTNER_KEY"
```

Once `status` is `generated`, the same response carries `viewUrl` — the report web page, **the deliverable you hand to your user** (§14). Read it straight off the status:

```
curl -sS "https://gamma.residencevertical.ro/api/partner/v1/reports/$REPORT_REQUEST_ID" \
-H "Authorization: Bearer $RV_PARTNER_KEY" | jq -r '.viewUrl, .viewExpiresAt'
```

Optionally, download the PDF yourself (§13):

```
curl -sS -o raport.pdf "https://gamma.residencevertical.ro/api/partner/v1/reports/$REPORT_REQUEST_ID" \
-H "Authorization: Bearer $RV_PARTNER_KEY"
```

If the link you stored has since expired, mint a fresh one:

```
curl -sS -X POST "https://gamma.residencevertical.ro/api/partner/v1/reports/$REPORT_REQUEST_ID"
-H "Authorization: Bearer $RV_PARTNER_KEY"
```

Reference integration and mock server — start before you have a key

A complete, runnable Node.js integration lives in the `samples/partner-api-node/` directory of [this documentation repository](#): a demo property page, the partner backend behind it, and **a local mock of this API**.

```
node mock-rv-api.js    # a local stand-in for this API, on :4010
node server.js         # the demo site + its backend, on :4000 → http://localhost:4000
```

Zero dependencies — Node.js ≥ 20 built-ins only, no `npm install`. The mock speaks the contract described here: Bearer auth, the error envelope and every `error.code`, `Idempotency-Key` replay/conflict, the daily cap, `X-RV-Request-Id`, a report that takes time to become `generated`, a real PDF download, **signed webhook delivery**, **signed report view links** (real HMACs, so a tampered or expired one is refused exactly as it would be here), **server-minted checkout links** (§2.1: `POST /checkout-links`), the public resolve and a stand-in `/c/<token>` checkout landing with the co-branding chip), a canned weekly settlement on `GET /settlements` (§16) and a canned referral ledger on `GET /referrals` whose money agrees with the `/me` `referral` block (§2.5, §2.6). Address-driven test hooks trigger a failed report, a slow report, `429 daily_cap_exceeded`, `503 maintenance`, `502 report_service_unavailable` and `502 geocoding_failed` on demand, so every branch of your error handling can be exercised in seconds — and `MOCK_VIEW_LINK_TTL_SECONDS` compresses a 30-day link lifetime into a few seconds so you can see the expired state your user would get.

So your developers can build and test the whole flow **before we issue an API key** — and without spending real reports. When the key arrives, point the same code at this environment with two environment variables (`RV_API_BASE_URL`, `RV_API_KEY`); no code changes. The sample's `lib/webhookSignature.js` is a copy-ready implementation of §10.3.

7. Endpoint reference

All requests and responses are JSON (`Content-Type: application/json`) except the PDF download and the OpenAPI document. Timestamps are ISO-8601 UTC (`2026-08-18T10:00:00Z`). Money is expressed in **cents of RON** (bani): `5000` = 50 lei.

7.1 `POST /api/partner/v1/reports` — create a report request

Headers: `Authorization`, `Content-Type: application/json`, optional `Idempotency-Key` (≤ 128 characters, strongly recommended — see §9).

Request body:

```
{
  "address": {
    "street": "Strada Turda",
    "streetNumber": "94",
    "city": "București",
    "county": "București",
    "postalCode": "011332"
  },
  "coordinates": { "lat": 44.4600, "lng": 26.0500 },
  "propertyType": "apartment",
  "residentialComplex": "Ansamblul X",
  "adUrl": "https://www.imobiliare.ro/...",
  "customer": { "email": "buyer@example.com", "name": "Ion Popescu" },
  "externalReference": "lead-84213",
  "webhookUrl": "https://partner.example/rv-hook"
}
```

Field	Required	Rules
address.street	yes	Street name without the number (e.g. Strada Turda , Bulevardul Unirii).
address.streetNumber	yes	House/building number, e.g. 94 , 12B , 229–231 .
address.city	yes	Locality, e.g. București , Cluj-Napoca , Chiajna .
address.county	no	County (județ).
address.postalCode	no	Postal code.
coordinates.lat / coordinates.lng	no	WGS-84; lat in [-90, 90], lng in [-180, 180]. If omitted we geocode the address (§12).
propertyType	yes	apartment or house .
residentialComplex	no	The name of the residential complex / ansamblu (e.g. Cosmopolis), never an address.
adUrl	no	http(s) URL of the property listing (Imobiliare.ro, Stora.ro, ...).
customer.email	no	RFC-shaped email. If present, the buyer receives our standard delivery email with the PDF (§12).
customer.name	no	Buyer's name.
externalReference	no	Your own lead/order id, ≤ 128 chars; echoed in responses and webhooks.
webhookUrl	no	Per-request override of your account's webhook URL (same validation, §10).

Response `202 Accepted`:

```
{
  "reportRequestId": "0d5f7c1e-8f2b-4c1a-9a1e-3b7d2f6a1c00",
  "reportId": "3f0a6c2e-1b4d-4e8f-9c7a-2d5b8e1f4a10",
  "status": "processing",
  "failureReason": null,
  "createdAt": "2026-08-18T10:00:00Z",
  "generatedAt": null,
  "downloadUrl": null,
  "externalReference": "lead-84213",
  "address": { "street": "Strada Turda", "streetNumber": "94", "city": "București", "county":
  "coordinates": { "lat": 44.46, "lng": 26.05 },
  "propertyType": "apartment",
  "estimatedReadySeconds": 240,
  "statusUrl": "https://gamma.residencevertical.ro/api/partner/v1/reports/0d5f7c1e-8f2b-4c1a-
}
```

The body is the §7.2 status representation plus two creation-only fields: `estimatedReadySeconds` and `statusUrl`. `reportRequestId` is the id you use for every subsequent call. `reportId` is our internal report id — keep it for support conversations. When the same `Idempotency-Key` is replayed with an identical body the endpoint returns `200 OK` with this same `202` body shape (`statusUrl` and `estimatedReadySeconds` included) for the existing request, instead of creating a second report.

A fresh `202` never carries `viewUrl` — the report does not exist yet. A `200` replay of a request that has since been generated does, because it is the same status representation (§7.2).

7.2 `GET /api/partner/v1/reports/{reportRequestId}` — status

```
{
  "reportRequestId": "0d5f7c1e-8f2b-4c1a-9a1e-3b7d2f6a1c00",
  "reportId": "3f0a6c2e-1b4d-4e8f-9c7a-2d5b8e1f4a10",
  "status": "generated",
  "failureReason": null,
  "createdAt": "2026-08-18T10:00:00Z",
  "generatedAt": "2026-08-18T10:03:12Z",
  "downloadUrl": "https://gamma.residencevertical.ro/api/partner/v1/reports/0d5f7c1e-8f2b-4c1
  "viewUrl": "https://gamma.residencevertical.ro/raport/0d5f7c1e-8f2b-4c1a-9a1e-3b7d2f6a1c00?",
  "viewExpiresAt": "2026-09-17T10:03:12Z",
  "externalReference": "lead-84213",
  "address": { "street": "Strada Turda", "streetNumber": "94", "city": "București", "county":
  "coordinates": { "lat": 44.46, "lng": 26.05 },
  "propertyType": "apartment"
}
```

Optional address fields you did not send come back as `null` (e.g. `"county": null`).

Field	Values
status	processing → generated or failed . processing lasts until the report is ready — usually ~4 minutes, but it may persist while our internal retries run (up to ~6 hours in the worst case). failed is returned only once the report is terminally failed (no further internal retries) and is never billed. In rare, operator-initiated recoveries a failed request can still move to generated later; see §10.4 and §15.
failureReason	null while processing/generated; otherwise report_failed , report_cancelled or report_service_error .
generatedAt	Set only when status is generated .
downloadUrl	Non-null only when status is generated ; it is the §7.3 URL and requires your key.
viewUrl	The report web page — a signed, time-limited link you can hand straight to your end user (§14). Present only when status is generated : on a processing or failed report the key is absent , not null (and it is absent on every report if view links are switched off for an environment, which leaves the PDF untouched). A fresh link is minted on every read, so the one you just received always has the full lifetime ahead of it.
viewExpiresAt	When that viewUrl stops working (30 days after it was minted, unless your account is configured otherwise). Present alongside viewUrl only.

While a request is still processing we refresh its state from the report engine before answering (at most once every few seconds), so polling this endpoint always reflects the current state.

7.3 GET /api/partner/v1/reports/{reportRequestId}/pdf — download the PDF

Returns the report bytes:

```
HTTP/1.1 200 OK
Content-Type: application/pdf
Content-Disposition: attachment; filename="raport-imobiliar-0d5f7c1e-....pdf"
```

Before the report is generated the endpoint answers 409 report_not_ready ; an unknown or foreign id answers 404 not_found . See §13 for handling guidance.

This endpoint accepts **either** credential: your Authorization: Bearer rvp_... key (the server-to-server call you make), or a report view token in ?t= — which is what the "Descarcă PDF" button on the report web page uses, so a browser can download the PDF without ever holding your key. Send one or the other. An Authorization header always wins, a request with neither is the same 401 unauthorized as before, and an invalid or expired ?t= answers 401 invalid_or_expired_view_token . **You** normally use the Bearer form; the token form exists for the page.

7.4 POST /api/partner/v1/reports/{reportRequestId}/view-link — mint a fresh web link

Returns a **new** signed, time-limited link to the report's web page:

```
{
  "reportRequestId": "0d5f7c1e-8f2b-4c1a-9a1e-3b7d2f6a1c00",
  "viewUrl": "https://gamma.residencevertical.ro/raport/0d5f7c1e-8f2b-4c1a-9a1e-3b7d2f6a1c00",
  "viewExpiresAt": "2026-09-17T10:03:12Z"
}
```

You do not need this to *get* a link — a generated report already carries one on §7.2 and in the `report.generated` webhook. Call it when the link you stored has **expired** (`viewExpiresAt` is in the past), or whenever you prefer minting on demand over reading the status.

- The lifetime is your account's view-link TTL (30 days unless we configured another value for you).
- **Minting a new link does not revoke older ones** — each runs out on its own schedule.
- Only a generated report has a page: before that the answer is `409 report_not_ready`. An unknown or foreign id answers `404 not_found`.

7.5 The two endpoints the page itself calls

You normally never call these — **the browser does**, when your user opens a `viewUrl`. They are documented because they are part of the contract, and because you may want to build your own page on top of the same data.

Endpoint	What it is
GET <code>/reports/{reportRequestId}/view-data?t=...</code>	The JSON the page renders: the full premium report payload plus the address and generation date. No API key — the signed <code>t</code> token is the credential. Never send your key here.
GET <code>/reports/{reportRequestId}/pdf?t=...</code>	The same PDF as §7.3, authenticated by the token instead of your key — the page's download button.

```
{
  "reportRequestId": "0d5f7c1e-8f2b-4c1a-9a1e-3b7d2f6a1c00",
  "generatedAt": "2026-08-18T10:03:12Z",
  "address": { "street": "Strada Turda", "streetNumber": "94", "city": "București", "county": "Cluj" },
  "propertyType": "apartment",
  "partnerName": "Portal Imobiliar SRL",
  "report": {
    "premium_pdf_title": "Raport premium – Strada Turda 94",
    "location": { "strada": "Strada Turda", "numar": "94", "localitate": "București" }
  }
}
```

Two things to know if you do build your own page:

- `report` is the same document the PDF is rendered from — location, area intelligence, market intelligence, developer profile, section narratives, warnings, quality review. It is an **evolving document**: new keys are added over time, so ignore what you do not recognise rather than failing on it.

- The payload deliberately contains **nothing commercial** — no customer email, no payment id, no commission, no price. It is safe to render for an end user.

A rejected token and an unknown `reportRequestId` both answer `401 invalid_or_expired_view_token`, deliberately: these endpoints are reachable by anyone, so they must never confirm which report requests exist.

7.6 GET /api/partner/v1/reports?status=&limit=&offset= — list your requests

Newest first. `status` filters by `processing` / `generated` / `failed`; `limit` ≤ 100 (default 20); `offset` default 0.

```
{ "items": [ { "reportRequestId": "...", "status": "generated", "...": "..." } ], "count": 1, "limit": 100, "offset": 0 }
```

Each item has the §7.2 representation. `count` is the number of items **in this page** (not the total across all pages) — to walk your history, increase `offset` by `limit` until a page comes back with fewer than `limit` items.

7.7 GET /api/partner/v1/me — your account and usage

```
{
  "partnerId": "...",
  "name": "Agenția Exemplu",
  "slug": "agentia-exemplu",
  "environment": "test",
  "commissionPct": 15,
  "dailyReportCap": 100,
  "reportsToday": 3,
  "reportPriceCents": 5000,
  "currency": "RON",
  "webhookConfigured": true,
  "usageThisMonth": { "requested": 12, "generated": 11, "failed": 1, "commissionCents": 8250 },
  "referral": {
    "referralUrl": "https://gamma.residencevertical.ro/p/agentia-exemplu",
    "commissionPct": 15,
    "pendingCents": 750,
    "earnedUnpaidCents": 1500,
    "paidCentsAllTime": 2250
  }
}
```

`environment` is `test` or `live`. `dailyReportCap` is `null` when unlimited. Days and months are counted in the `Europe/Bucharest` time zone. Use this endpoint to verify a new key and to show your own usage dashboards.

`usageThisMonth` (including `commissionCents`) is an **informational running counter** for the current calendar month, covering your **API-mode** usage. Billing itself is settled **weekly** — the authoritative settlement history, with what each week's invoice actually contains, is `GET /settlements` (§16).

`referral` is your **referral-mode** block: the link to put on your site and your commission totals (pending / earned-but-unpaid / paid all-time). Field meanings in §2.5.

7.8 GET /api/partner/v1/referrals — your referral ledger

Documented with the rest of referral mode in §2.6: newest first, `limit` / `offset` pagination in the shared `{ items, count, limit, offset }` envelope, one row per attributed purchase with `status` `pending` / `earned` / `void`, `paidAt` set once its weekly payout is paid, and — for purchases attributed through a checkout link — your `externalReference` plus the `checkoutLinkId`. No buyer data is exposed.

7.9 POST /api/partner/v1/checkout-links — mint a checkout link (+ its public resolve)

Documented with referral mode in §2.1: a `201` with the `/c/<token>` URL to put behind your button, expiry chosen per request (`expiresInHours` 1..720, default 48), the link reusable until it expires. The companion `GET /checkout-links/{token}/resolve` is the **public** endpoint our checkout page calls itself (`410 checkout_link_expired` / `404 not_found`) — you never call it.

7.10 GET /api/partner/v1/openapi.yaml — OpenAPI 3 document

Public, no authentication, `Content-Type: text/yaml`. Generate clients from it or import it into your API tooling.

8. Error handling

Errors use one JSON shape:

```
{ "error": { "code": "validation_error", "message": "Request validation failed: address.street
```

`fields` is present only on validation errors; `message` joins every field message with `;` (e.g. `Request validation failed: address.streetNumber is required; propertyType must be one of: apartment, house`). Branch on `code` (and the `fields` keys), never on the text of `message`.

HTTP	code	Meaning	What to do
400	validation_error	Body failed validation; details in <code>message</code> / <code>fields</code> .	Fix the request. Do not retry unchanged.
401	unauthorized	Missing, malformed, unknown, revoked or expired key.	Check the <code>Authorization</code> header and the environment (<code>rvp_test_</code> vs <code>rvp_live_</code>). Contact support if the key should be valid.
403	partner_suspended	The key is valid but the account is suspended — for example after a settlement invoice remained unpaid past its due date (§16).	Contact ResidenceVertical; settle any outstanding invoice.
404	not_found	Unknown <code>reportRequestId</code> , or one that belongs to another account.	Check the id.
409	idempotency_conflict	The <code>Idempotency-Key</code> was already used with a different body.	Use a new key for a new request; replay the same body to get the original result.
409	report_not_ready	The PDF or a view link was requested while <code>status</code> is still <code>processing</code> . Only a generated report has either.	Keep polling / wait for the webhook, then retry.
401	invalid_or_expired_view_token	The <code>?t=</code> view token is missing, malformed, tampered with, expired, or was minted for a different report. The same answer is given for an unknown <code>reportRequestId</code> , so the endpoint cannot be used to discover ids.	Mint a fresh link with <code>POST /reports/{id}/view-link</code> (§7.4) and send your user the new URL. Never your API key — it is not accepted there.
410	checkout_link_expired	A checkout link was resolved after its expiry — answered by the public resolve endpoint to our own checkout page (§2.1),	Nothing on your side fails: the buyer sees a calm state that points back at you. Mint a fresh link for the lead.

HTTP	code	Meaning	What to do
		never to a call you make with your key.	
429	daily_cap_exceeded	Your account's daily report cap is reached. <code>Retry-After</code> gives the seconds until midnight (Europe/Bucharest).	Stop creating reports until the cap resets, or ask us to raise the cap.
429	(no JSON body)	Platform per-IP rate limit at the edge (where the environment applies one).	Back off (honour <code>Retry-After</code> if present) and retry; see §11.
502	geocoding_failed	We could not resolve the address to coordinates. Nothing was created and your <code>Idempotency-Key</code> is not consumed.	Check the address, or send <code>coordinates</code> explicitly and retry — the same key is fine.
502	report_service_unavailable	The report engine did not accept the request. A <code>failed</code> ledger row may exist for audit but it is not billable, and your <code>Idempotency-Key</code> is not consumed.	Retry after a short delay. Reusing the same <code>Idempotency-Key</code> creates a fresh attempt (it does not replay the failed one). If a webhook URL is configured you may still receive a <code>report.failed</code> event for the failed attempt — see §10.4.
503	partner_api_disabled	The Partner API is switched off on this environment.	Retry later; contact support if it persists.
503	maintenance	Report generation is paused for a deployment or maintenance window; <code>Retry-After: 60</code> .	Retry after <code>Retry-After</code> seconds.

Any 5xx without a JSON body comes from infrastructure in front of the API (load balancer, edge). Treat it as transient and retry with backoff.

9. Idempotency

Send an `Idempotency-Key` header (≤ 128 chars, unique per lead — for example your lead id or a UUID you store with the lead) on every `POST /reports`.

- Same key + same body → `200 OK` with the existing request (same body shape as the original `202`, see §7.1). No second report is created and nothing extra is billed.
- Same key + different body → `409 idempotency_conflict`.
- Keys are scoped to your account.
- A key is consumed only when we answer `202` (or replay it with `200`). A `POST` that answered `502 geocoding_failed` or `502 report_service_unavailable` does **not** consume the key: retrying with the same key creates a fresh attempt rather than replaying the failure.

This makes retries safe: if your request timed out and you do not know whether it reached us, replay it with the same key. Design your button so that one click produces at most one request with one key; if the user clicks again, replay the same key.

10. Webhooks

Webhooks are optional but recommended — they remove the polling loop and tell you the moment a report is ready. Configure an account-wide URL with `ResidenceVertical` (we set it on your account), or pass `webhookUrl` per request. When the request reaches a terminal state we `POST` to that URL.

The webhook URL is **snapshotted on each request when it is created** (the per-request `webhookUrl` if given, else the account URL at that moment). Changing or clearing the account URL later affects only **new** requests — requests already in flight keep delivering to the URL they were created with.

10.1 Payload

```
{
  "event": "report.generated",
  "deliveryId": "6c1b8a3e-...",
  "sentAt": "2026-08-18T10:03:42Z",
  "data": { "...the §7.2 status representation..." }
}
```

`event` is `report.generated` or `report.failed`. `data` is exactly what `GET /reports/{reportRequestId}` returns, so you can act on `data.status`, `data.viewUrl`, `data.externalReference` without a follow-up call.

A `report.generated` **delivery already contains a ready-to-use** `data.viewUrl` (plus `data.viewExpiresAt`), minted at send time with your account's full view-link TTL. A webhook-driven integration therefore needs **no extra API call to show the report**: store the link from the handler and your "Vezi raportul" button works immediately. Each retry of a delivery re-mints the link, so a link that reaches you from a retry is never shorter-lived. `report.failed` carries neither field — there is no report to show.

10.2 Headers

Header	Value
Content-Type	application/json
X-RV-Event	report.generated / report.failed
X-RV-Delivery-Id	Same as <code>deliveryId</code> . It identifies the event (one id per <code>reportRequestId</code> + <code>event</code>) and is stable across retries of that event — use it as your de-duplication key.
X-RV-Timestamp	Unix time in seconds when the request was signed.
X-RV-Signature	<code>v1=<hex></code> where <code><hex></code> = <code>HMAC-SHA256(webhookSecret, "<X-RV-Timestamp>.<raw body>")</code> .
User-Agent	ResidenceVertical-Webhooks/1.0

Your webhook secret (`whsec_...`) is given to you when the webhook is configured or rotated. It is shown once; store it like the API key.

10.3 Verifying the signature

Always verify before trusting a webhook. Use the **raw** request body bytes (not a re-serialised JSON object), compare in constant time, and reject requests whose timestamp is more than **300 seconds** away from your clock (replay protection).

```
=== "Node.js"
```

```

`js
const crypto = require("node:crypto");

// rawBody: Buffer (e.g. express.raw({ type: "application/json" }))
function verifyRvWebhook(rawBody, headers, secret) {
  const timestamp = headers["x-rv-timestamp"];
  const signature = headers["x-rv-signature"] || "";
  if (!timestamp || !/^\d+$/.test(timestamp)) return false;
  if (Math.abs(Date.now() / 1000 - Number(timestamp)) > 300) return false;

  const expected = crypto
    .createHmac("sha256", secret)
    .update(`${timestamp}.`)
    .update(rawBody)
    .digest("hex");
  const provided = signature.startsWith("v1=") ? signature.slice(3) : "";
  const a = Buffer.from(expected, "utf8");
  const b = Buffer.from(provided, "utf8");
  return a.length === b.length && crypto.timingSafeEqual(a, b);
}
`

```

=== "Python"

```

`python
import hashlib
import hmac
import time

def verify_rv_webhook(raw_body: bytes, headers: dict, secret: str) -> bool:
    timestamp = headers.get("X-RV-Timestamp", "")
    signature = headers.get("X-RV-Signature", "")
    if not timestamp.isdigit():
        return False
    if abs(time.time() - int(timestamp)) > 300:
        return False
    expected = hmac.new(
        secret.encode("utf-8"),
        timestamp.encode("utf-8") + b"." + raw_body,
        hashlib.sha256,
    ).hexdigest()
    provided = signature[3:] if signature.startswith("v1=") else ""
    return hmac.compare_digest(expected, provided)
`

```

=== "PHP"

```

` ``php
<?php
function verifyRvWebhook(string $rawBody, array $headers, string $secret): bool
{
    // $headers keys lower-cased, e.g. from getallheaders() + array_change_key_case()
    $timestamp = $headers['x-rv-timestamp'] ?? '';
    $signature = $headers['x-rv-signature'] ?? '';
    if (!ctype_digit($timestamp)) {
        return false;
    }
    if (abs(time() - (int) $timestamp) > 300) {
        return false;
    }
    $expected = hash_hmac('sha256', $timestamp . '.' . $rawBody, $secret);
    $provided = str_starts_with($signature, 'v1=') ? substr($signature, 3) : '';
    return hash_equals($expected, $provided);
}

// Usage (plain PHP):
// $rawBody = file_get_contents('php://input');
// $headers = array_change_key_case(getallheaders(), CASE_LOWER);
// if (!verifyRvWebhook($rawBody, $headers, $secret)) { http_response_code(400); exit; }
` ``

```

10.4 Delivery, retries and idempotent handling

- **Success** = your endpoint answers any `2xx` within 10 seconds. Answer quickly (persist the event, then process asynchronously); do the PDF download outside the webhook handler. **Redirects are not followed** — a `3xx` answer counts as a failed attempt, so point us at the final URL.
- **Retries**. If delivery fails (non-2xx incl. 3xx, timeout, connection error) we retry with growing delays — approximately 1 min, 5 min, 15 min, 1 h and 3 h after the previous attempt — i.e. **up to 5 retries after the initial attempt (6 attempts in total)**, the last one roughly **4.5 h** after the first. After that the delivery is marked failed and not retried; the report itself is unaffected and still available via `GET /reports/{id}` and `/pdf`.
- **Duplicates are possible** (a retry after a slow 2xx, for example). `deliveryId` / `X-RV-Delivery-Id` is the same on every retry of the same event, so de-duplicate on it, and treat `data.status` as the truth for the request.
- **Handle events idempotently by** `(reportRequestId, event)`. Each request normally produces exactly one event. In rare, operator-initiated recoveries a request that already produced `report.failed` can later be generated, in which case a `report.generated` event follows the `report.failed` one (with its own `deliveryId`); the later event wins — apply `data.status` rather than rejecting the event because you already saw one for that request.
- `report.failed` **after a 502**. If your `POST /reports` was answered `502` `report_service_unavailable` and a webhook URL is configured, you may still receive a `report.failed` event (`failureReason: report_service_error`) for that failed attempt — the `502` body carries no `reportRequestId`, so correlate it through `data.externalReference`. It is informational: nothing was billed and your `Idempotency-Key` is free to reuse (§9).

- **Fallback.** Webhooks are a notification, not the system of record. If you have not received an event ~6 minutes after creating a request, poll `GET /reports/{id}`.
- **URL rules.** The URL must be reachable from the public internet; private, loopback, link-local or multicast targets are rejected. On the production environment the URL must be `https://`; on the test environment `http://` is tolerated for local tunnels, but use HTTPS whenever you can. A URL that fails validation is rejected at configuration time (or with `400 validation_error` on a per-request `webhookUrl`) — this covers literal IPs and `localhost`. A **hostname** that resolves to a private address passes configuration and fails only at delivery time (the delivery is marked failed, without a `400`): if your webhooks never arrive, check what your hostname resolves to from the public internet and ask support to read the delivery error on the request.

11. Daily cap and rate limits

- **Daily report cap.** Each account has a per-day cap on report requests (default 100, configured by `ResidenceVertical` for your agreement; days in Europe/Bucharest). Reaching it returns `429 daily_cap_exceeded` with `Retry-After`. Check `reportsToday` / `dailyReportCap` on `GET /me` if you need to display headroom.
- **Platform rate limits.** Depending on the environment, the platform may apply per-IP limits at the edge (on the order of 40 requests/second across all endpoints). Normal integrations never approach this; a polling loop that runs one request every 5–10 s per active report is fine.
- **Design your button flow so one click creates at most one request.** Disable the button after the click, use one `Idempotency-Key` per lead, and never fire `POST /reports` from a page reload. Repeated identical requests are safe (they replay), but repeated requests with new keys create — and bill — new reports.

12. Data you must send (and what happens with it)

- **Address.** `street` (name only), `streetNumber` and `city` are required. The more precise the address the better the report; include `postalCode` and `county` when you have them. The number matters — much of the report (developer attribution, building permits, seismic registry) is resolved at building level, so a street without a number produces a much weaker report or a validation error.
- `propertyType`. `apartment` or `house`. Sending the wrong type changes the analysis vocabulary and checks (e.g. a house report covers land/cadastre topics that an apartment report does not).
- **Coordinates (optional).** If you already have the pin (from your own map/geocoder), send `coordinates` — it is used as-is and is the most reliable input. If omitted, we geocode `"<street> <streetNumber>, <city>[, <postalCode>]"` server-side. Geocoding can fail for unusual or new addresses (`502 geocoding_failed`, nothing created); in that case send coordinates.
- `residentialComplex` (**optional**). Only when the property is part of a named complex and you know the **name** (e.g. `Cosmopolis`, `One Floreasca City`). Never put an address, a street or a building number in this field — address-shaped values are ignored, and a wrong name can attach the wrong developer to the report.

- `adUrl` **(optional)**. The public listing URL. When present we extract the listing's own data (price, surface, photos) and cross-check it, which noticeably improves the report.
- `customer.email` **semantics**.
 - If provided, the end buyer **also receives our standard delivery email** with the PDF attached (plus our standard "report in progress" notice when generation is slow), exactly as a direct customer would. You may still serve the PDF yourself.
 - If omitted, **nothing is emailed by ResidenceVertical** and you are responsible for putting the report in front of your user — normally the `viewUrl` button (§14), plus the PDF (§13) if your flow also needs the file.
- `externalReference` **(optional)**. Your lead/order id, echoed back in every response and webhook — the easiest way to correlate our `reportRequestId` with your data.

13. PDF handling

The PDF is a **secondary export** — your user already gets it from the report page (§14); fetch it yourself only if your flow needs the file server-side. That means you need this section only if you want the PDF **yourself** — to archive it, attach it to your own email, or serve it from your own route. If all you want is to show your user the report, the web page of §14 already offers them the download and you can skip this.

- The download endpoint returns raw PDF bytes with `Content-Type: application/pdf` and a suggested filename `raport-immobilier-<reportRequestId>.pdf`. Read the whole body before writing the file (a multi-page report with images is a few megabytes).
- **Do not hotlink** `downloadUrl` **from your website**. The URL requires your API key, so it cannot be opened by a browser without exposing the key. Download it in your backend and serve it to your user through your own authenticated route (or attach it to your own email). The link that *is* safe to put in front of a browser is `viewUrl` (§14) — never `downloadUrl`.
- **Cache it**. Store the PDF once after `generated`; the content does not change. Serve every later view from your storage rather than re-downloading it from us.
- Serve it with `Content-Disposition: attachment` (or inline in a viewer) and your own filename if you prefer.

14. Show the report as a web page

The web page is the deliverable — the report your user is meant to read. Every generated report has one: the same interactive report ResidenceVertical's own buyers read, section by section — location and area intelligence, developer attribution, seismic registry, building permits, comparable prices, transport, schools, public signals — with the PDF download offered on the page itself, so this one link covers everything your user needs.

You reach it through `viewUrl`, which arrives with the report (§7.2, and in the `report.generated` webhook) and looks like this:

```
https://gamma.residencevertical.ro/raport/0d5f7c1e-8f2b-4c1a-9a1e-3b7d2f6a1c00?t=178943040000
```

Hand it to your user as-is. It is designed to be put in front of a browser:

- **No login and no API key.** The signature in `t` is the whole credential. It is bound to that one report, so it opens nothing else, and it expires.
- **It does not sell to your user.** The page carries the ResidenceVertical mark (we authored the report) but **no purchase CTAs and no navigation into our funnel** — nothing invites your user to buy from us instead of from you. Its header is the address, the generation date, the property type and a discreet *"Raport furnizat prin <your account name>"* line.
- **Nothing commercial is exposed.** No customer email, no payment id, no commission, no price.

Lifetime, and what your user sees when it lapses

A link lives **30 days** by default (ResidenceVertical can configure another value for your account; `viewExpiresAt` always tells you the exact instant). After that the page shows a short, calm message in Romanian — *"Linkul nu mai este valid. Solicită un link nou de la partenerul care ți l-a trimis."* — which points your user back at **you**, not at our support.

So make sure they can get a new one: `POST /reports/{id}/view-link` (§7.4) mints a fresh link at any time, and older links keep working until their own expiry.

The practical recipe

1. **Store** `viewUrl` and `viewExpiresAt` next to your order, the moment the report is generated — from the status read or straight from the webhook. Never build the URL yourself; it is signed.
2. **Put it behind a button** ("Vezi raportul"). Open it in a new tab; your backend is not in the path.
3. **Re-mint on demand rather than caching forever.** When a user comes back after the link has lapsed, call §7.4 and give them the new URL — do not hide the report, and do not store a dead link.
4. **Do not proxy the page** through your own server, and do not try to strip the token: the token *is* the access.

If you would rather build the page yourself, §7.5 documents the two endpoints it is made of.

!!! note "On the test environment" `gamma.residencevertical.ro` sits behind a team access wall, with two carve-outs that matter to you: everything under `/api/partner` (the API you integrate against) and the page route `/raport/...` (so a gamma `viewUrl` opens in a browser like it will in production). Both behave exactly as on production, where there is no wall at all.

If a gamma `viewUrl` ever redirects you **to** `residencevertical.cloudflareaccess.com` instead **of** showing the report, the carve-out has **not** been applied **to** that environment yet — tell us **and** we will **enable** it.

15. Timing guidance

- Reports usually complete within **~4 minutes** (`estimatedReadySeconds: 240`). Show the user a "your report is being generated" state rather than blocking the request.
- **Poll** `GET /reports/{reportRequestId}` every **5–10 seconds** for up to **6 minutes**. Honour any `Retry-After` header. If the request is still `processing` after 6 minutes, it is being retried on our side: keep it in a slower background poll (e.g. every few minutes) — it reaches `generated` or `failed` within at most **~6 hours**. Contact support with the `reportRequestId` if it does not resolve by then.
- **Prefer webhooks** for production volume: zero polling, and the event payload already carries a ready-to-use `viewUrl` (plus `downloadUrl` for the optional PDF export, §10.1) — so the handler can show the report without a single follow-up call.
- A `failed` status means the report is **terminally** failed on our side — we do not retry it further, and it is **not billed**; you may create a new request (new `Idempotency-Key`) or surface the failure to your user. Only in rare, operator-initiated recoveries can a `failed` request still become `generated` later (you would then also get a `report.generated` webhook, §10.4) — if you store our status, let a later `generated` overwrite an earlier `failed` . A recovered report is billed in the **next** weekly settlement after the recovery (§16).

16. Commission and settlement (API mode)

This section is the money model for **API mode** — reports your backend creates through `POST /reports` . Referral-mode commissions work the other way around (we pay you) and are covered in §2.4.

The commercial model in one paragraph: you sell to your own user, at your own price, and you pay ResidenceVertical weekly for the reports we generated for you — with your commission given as a visible negative line on our invoice. The Partner API never charges your end user and never invoices them; the commercial relationship between you and your users is entirely yours.

The building blocks:

- **The report list price is 50 lei** (`reportPriceCents: 5000` , `currency: RON`).
- **Only successfully generated reports bill** (`status: generated`). Failed or cancelled requests are **never billed** — a `failed` request costs you nothing.
- **Your commission** (`commissionPct` , visible on `GET /me` , configured by ResidenceVertical according to your agreement — default 15%) is your margin. Per report it is `round(reportPriceCents × commissionPct / 100)` cents, snapshotted at request time, so a later percentage change never rewrites history.

The weekly cycle

- A settlement week runs **Monday 00:00 → Sunday 24:00, Europe/Bucharest**.
- **Every Monday** we close the previous week for your account: the `N` reports that reached `generated` in that week settle together as one settlement — `gross = N × 50 lei` , `net = gross - commission` . A week with zero generated reports produces no settlement and no invoice. Each report settles **exactly**

once.

- We issue **one invoice** for the settlement, with two lines — the commission is not deducted silently, it is a **visible negative line**, which is how your revenue share is given:

```
1) Rapoarte premium generate prin Partner API, perioada dd.MM-dd.MM.yyyy: N buc x 50,00 l
2) Comision partener <pct>: X,XX lei
```

The invoice total is the **net** you owe.

- **Payment terms are 7 days** from the invoice date.
- Alongside the invoice you receive a **statement email** (subject `ResidenceVertical – Decont Partner API dd.MM-dd.MM`), sent to your billing email (or your contact email when no billing email is configured). It contains the settlement totals — report count, gross, commission, net — plus a table of every report in the settlement: `reportRequestId`, your `externalReference`, the address, when it was generated, its price and its commission (very large settlements list the first 100 reports). Reconcile it against your own ledger and `GET /reports` (§7.6). When you also had **referral** activity that week, the same email carries your referral section — earned count and total, the payout period, and the amount to invoice us for (§2.4).
- **Late recoveries roll into the next week.** In the rare operator-initiated recovery where a request you saw as `failed` later becomes `generated` (§10.4, §15), that report is picked up by the **next** weekly close after the recovery — it is never squeezed into a week that was already settled, and it still settles exactly once.
- **Unpaid invoices can lead to account suspension.** If an invoice remains unpaid past its due date, ResidenceVertical may suspend your account — every call then answers `403 partner_suspended` (§8) — until the balance is settled. If something about an invoice looks wrong, contact us before the due date rather than withholding payment silently.

For invoicing we need your **billing details** (company name, CUI, trade-register number, address, billing email). They are configured on your account by ResidenceVertical at onboarding — tell us whenever they change; an incomplete billing profile delays the invoice (the settlement waits, it does not disappear).

`GET /api/partner/v1/settlements` — your settlement history

Newest first. `limit` ≤ 100 (default 20), `offset` default 0 — the same pagination and envelope as `GET /reports` (§7.6).

```

{
  "items": [
    {
      "settlementId": "9c2f4b7a-1e3d-4a5c-8b6f-0d9e2a4c6b1e",
      "periodStart": "2026-08-10",
      "periodEnd": "2026-08-16",
      "reportsCount": 12,
      "grossCents": 60000,
      "commissionCents": 9000,
      "netCents": 51000,
      "currency": "RON",
      "status": "invoiced",
      "invoiceNumber": "RVTP 0042",
      "issuedAt": "2026-08-17T06:00:00Z",
      "dueAt": "2026-08-24T06:00:00Z",
      "paidAt": null
    }
  ],
  "count": 1,
  "limit": 20,
  "offset": 0
}

```

Field	Meaning
periodStart / periodEnd	The Monday and the Sunday of the settled week (dates, Europe/Bucharest).
reportsCount	Generated reports settled in this week.
grossCents / commissionCents / netCents	List value of those reports, your commission, and what you owe (<code>net = gross - commission</code>). Cents of RON.
status	<code>open</code> (week closed, invoice not issued yet) → <code>invoiced</code> (invoice issued, awaiting payment) → <code>paid</code> . <code>void</code> = cancelled by ResidenceVertical; its reports are released and re-settled by a later close.
invoiceNumber	The invoice series + number (e.g. <code>RVTP 0042</code>); <code>null</code> while no invoice exists.
issuedAt / dueAt / paidAt	Invoice issue timestamp, its due date (7 days later), and when the payment was registered; <code>null</code> where not applicable.

Use this endpoint to reconcile our invoices against your own numbers. `usageThisMonth` on `GET /me` remains a monthly informational counter; the settlement rows are the billing truth.

17. Support and escalation

- Support: `support@residencevertical.ro`.
- Always include: environment (test/live), `reportRequestId` (and `reportId` if you have it), the `X-RV-Request-Id` of the failing call, the timestamp (UTC), and the HTTP status
- `error.code` you received. For webhook issues add the `deliveryId`.
- Key leaks, suspected fraud or an urgent production outage: say so in the subject line and we will revoke/rotate keys and investigate with priority.
- Roadmap items not shipped yet: partner self-service portal, embeddable JavaScript widget, and co-branding of the report web page. Tell us if any of these would unblock your integration.

18. Changelog

Version	Date	Notes
v1.4	2026-08-21	Checkout links + co-branding. Referral mode is restructured into two tiers (§2). The recommended tier: <code>POST /api/partner/v1/checkout-links</code> (§2.1) mints a server-attributed <code>/c/<token></code> checkout URL for one property — the buyer lands on our checkout with the address (and, optionally, their email) prefilled; links are reusable until expiry (<code>expiresInHours</code> 1..720, default 48 h); a public <code>GET /checkout-links/{token}/resolve</code> backs the page (new error code <code>checkout_link_expired</code>). Per-lead conversion tracking: <code>GET /referrals</code> items now carry <code>externalReference</code> + <code>checkoutLinkId</code> , copied from the link at attribution time (§2.6). Both tiers show a discreet co-branding chip on the checkout — <i>"Comandă prin partener: <name>"</i> . The <code>/p/<your-slug></code> link stays as the zero-code option (§2.2). Additive only — every v1.3 call and field is unchanged.
v1.3	2026-08-21	Referral mode — the recommended integration. A second partner mode (§2), alongside API mode: you link your users to our checkout via <code>/p/<your-slug></code> (with optional <code>street</code> / <code>number</code> / <code>city</code> prefill parameters), the user pays the normal 50 lei on our checkout with our standard invoice and delivery, and you earn <code>commissionPct</code> per generated report — paid to you weekly by SEPA transfer to your IBAN (gross, B2B; you invoice us for the commission). New: the <code>referral</code> block on <code>GET /me</code> (§2.5) and <code>GET /api/partner/v1/referrals</code> (§2.6); the weekly statement email gains a referral section. Both modes run simultaneously on one account at the same percentage (§2.7). Additive only — every v1.2 call and field is unchanged.
v1.2	2026-08-21	Weekly settlements. The billing model is now documented end-to-end (§16): you pay ResidenceVertical weekly for the previous week's generated reports (Monday close, Europe/Bucharest) at the 50 lei list price, with your commission as a visible negative line on the invoice ; 7-day payment terms; a statement email accompanies every settlement; failed reports never bill and late-recovered reports roll into the next week. New endpoint <code>GET /api/partner/v1/settlements</code> (§16) lists your settlement history. Additive only — every v1.1 call and field is unchanged.
v1.1	2026-08-20	Report web view. A generated report now carries <code>viewUrl</code> + <code>viewExpiresAt</code> — a signed, time-limited link to the interactive report page you can hand straight to your end user (§14), also delivered inside the <code>report.generated</code> webhook. New <code>POST /reports/{id}/view-link</code> (§7.4) mints a fresh link; the page's own endpoints are <code>GET /reports/{id}/view-data?t=</code> and <code>GET /reports/{id}/pdf?t=</code> (§7.5). New error code <code>invalid_or_expired_view_token</code> . Additive only — every v1.0 call and field is unchanged.
v1.0	2026-08-18	Initial release: <code>create/status/list/pdf/me</code> endpoints, idempotency, signed webhooks, daily cap, OpenAPI document.